

Uber

Functional requirements:

user should be able to book a cab based on his preference,
rider should be able to accept/reject any rider based on his preference,
user should be able to search for locations based on his preference,

Non-functional requirements:

Data-partitioning,
load balancing,
fault tolerance,
vertical scalability,
horizontal scalability,
low latency,
high throughput,

CAP theorem (Here we need to prioritise Strong Consistency in order to avoid double booking/double payment & for search scenario we need to prioritise 99.99% uptime i.e Availability>>Consistency)

Core Entities:

book,
user,
payment,
history

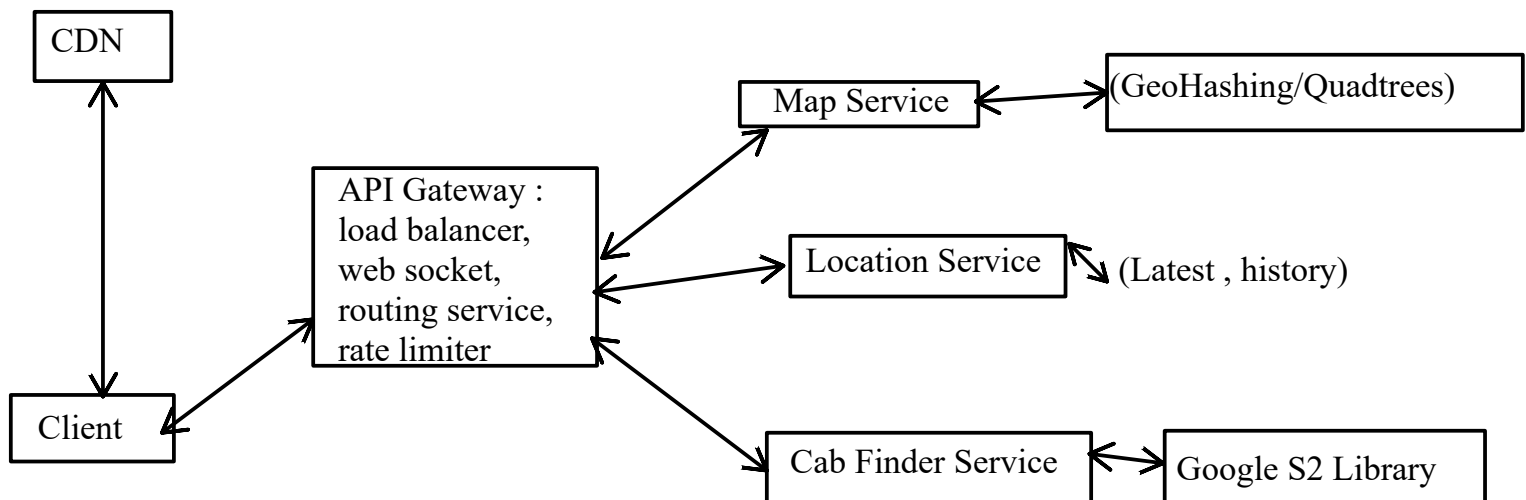
API :

POST:/booking

```
{  
  userID,  
  bookingID,  
  bookingStatus  
}
```

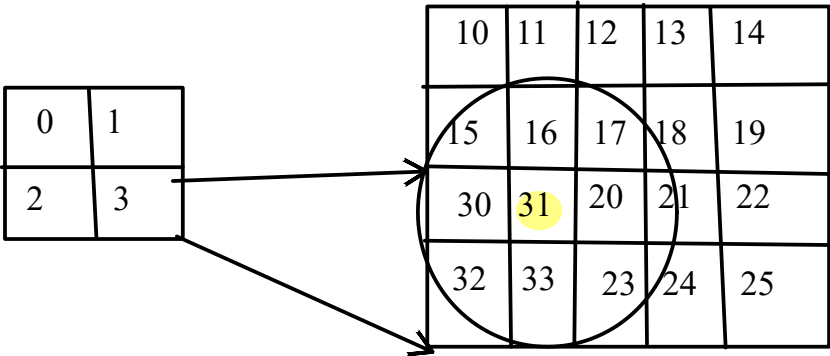
GET:/search/{locationID}

HLD :



Map Service (GeoHashing/Quadtrees)

(lat,long) -> chunkID



Google S2 library

chunkID -> (riders)

Consistent hashing, Gossip protocol

